Programmation Orientée Objet

Partie 1 : Standard Template Library

David Parello
david.parello@univ-perp.fr

Université Perpignan Via Domitia

Outline

Qu'est ce que la STL ?

Les conteneurs

Type auto

Fonctions Lambdas

Algorithmes

Qu'est ce que la STL ?

- La STL est une partie importante de la bibliothèque standard du C++ développée par Hewlett-Packard puis Silicon Graphics
- 1994 : adoptée par le comité de standardisation du C++ pour être intégrée au langage
- Plus particulièrement, la bibliothèque standard du C++ est formée de trois grandes composantes :
  - La STL
  - Les autres outils de la bibliothèque standard du C++ : flux d'entrée/sortie, ...
  - les bibliothèques du langage C.
- Vous pouvez consulter la liste des fichiers d'en-tête de la bibliothèque standard dans votre installation (généralement dans *usr/include/c++*) ou bien par ce lien.

Qu'est ce que la STL ?

- La STL propose un grand nombre d'objets et méthodes standards pour le C++
- des structures de données évoluées (tableaux, listes chaînées, vecteurs, piles, ...)
- des algorithmes efficaces sur ces structures de données (ajout, recherche, parcours, suppression, ...).
- Plus précisément, la STL propose principalement trois types d'éléments :
  - les conteneurs de base ou évolués => permet de stocker les données (vecteurs, listes, . . .)
  - les itérateurs => permet de parcourir les conteneurs,
  - les algorithmes → permettent de travailler sur les conteneurs via les itérateurs.

Utilité des templates

- Le C++ autorise la surcharge de symboles : un symbole peut avoir plusieurs significations différentes en fonction du contexte.
- surcharge d'opérateurs : a « n'aura pas la même signification si a est un entier ou un flux.
- surcharge de fonctions
- La résolution de conflits s'effectue à la compilation.
- Bien qu'utile, la surcharge de certains symboles peut conduire dans certains cas à une duplication importante du code. Par exemple :

```
1  int minimum ( int a , int b )
2  {
3     return a < b ? a : b ;
4  }
5  float minimum ( float a , float b )
6  {
7     return a < b ? a : b ;
8  }
9  // etc ...
```

Qu'est ce qu'un template ?

- Un patron ou template est un modèle à partir duquel des classes ou des fonctions pourront être générées automatiquement par le compilateur en fonction d'une série de paramètres.
- une classe (ou structure) paramétrée par le type d'un de ses attributs
- une fonction paramétrée par le type d'un de ses paramètres
- Un template est indiqué grâce à la mention template<typename T> qui précise que l'on a affaire à un patron dans lequel apparaît un paramètre de type T.

```
1  template <typename T> T minimum (T a , T b )
2  {
3     return a < b ? a : b ;
4  }
5  int main ( ) {
6  int n = 4 , p = 12;
7  float x = 2.5 , y = 3.25;
8  std::cout << " min ( n , p ) = " << minimum ( n , p ) << std::endl ;
9  std::cout << " min ( x , y ) = " << minimum ( x , y ) << std::endl ;
10 }
```

Qu'est ce qu'un template ?

- Un patron de fonction peut également comporter un ou plusieurs paramètres de type, chacun devant être précédé du mot clé typename.

```
1 template <typename T , typename U> f c t (T a , T b , U c )
2 {
3 ...
4 }
```

- Ces paramètres peuvent intervenir à n'importe quel endroit de la définition du patron :
  - dans l'en-tête ;
  - dans les déclarations de variables locales ;
  - dans les instructions exécutables comme **`sizeof()`**.

Lors de la compilation, le compilateur crée les instances de fonctions utilisées à partir du **`template`** fourni.

Identification des paramètres de type d'une fonction patron

- Avant de générer l'instance de la fonction à créer, le compilateur doit procéder à l'identification des types appelés et s'assurer qu'ils correspondent bien au modèle décrit dans le patron.
- La règle prévue par le C++ est que les types doivent être exactement identiques (les qualificateurs comme const inclus).

```
1  template <typename T> T minimum (T a , T b ) ;
2  int n; char c; unsigned int q;
3  const int ci1 = 10 , ci2 = 12;
4  int t[10];
5  int *adi ;
6  minimum ( n, c ) ; // Erreur
7  minimum ( n, q ) ; // Erreur
8  minimum ( n, c i 1 ) ; // Erreur
9  minimum ( ci1, ci2 ) ; // minimum (const int, const int)
10 minimum ( t, adi ) ; // minimum( int *, int *) compile car t
11 // est converti en int * avant l'appel
```

Identification des paramètres de type d'une fonction patron

- Le C++ autorise cependant la spécification d'un ou plusieurs types de paramètres de sorte à forcer la conversion.

```
1 template <typename T> T minimum (T a , T b );
2 template <typename T , typename U> f c t (T a , T b , U c );
3 int n; unsigned int q; char c;
4 minimum<int>(c, n ); // force la conversion de c en int .
5 minimum<char>(q, n ) ; // force la conversion de q et n en char
6 fct <float , int >(n , q , c ) ; // force l'utilisation de float pour T
7 // et de int pour U
8 fct <float >(n , q , c ) ; // force l'utilisation de float pour T
9 // U est defini par les regles habituelles
```

- Les conversions d'ajustement de type telles que **int**->=float= ne peuvent se faire que suivant une certaine hiérarchie :
  - **int** -> **long** -> **long long** -> **float** -> **double**
- Les type **short**, **char** ou **bool** sont convertis en **int** avant de suivre la hiérarchie ci-dessus.

Les conteneurs de la STL

- Les conteneurs sont des structures de données abstraites permettant de contenir d'autres objets.
- paramétrés par le type des éléments qu'ils contiennent.
- Les principaux conteneurs de la STL sont :
  - les paires → `pair`
  - les vecteurs (≈ tableaux dynamiques) → `vector`
  - les listes doublement chaînées → `list`
  - les tableaux doublement dynamiques → `deque`
  - les ensembles ordonnés (au sens mathématique) → `set`
  - les tables associatives ordonnées (dictionnaires) → `map`
  - les piles (structure LIFO) → `stack`
  - les files (structure FIFO) → `queue`

Les conteneurs de la STL

- Sur tous les conteneurs un ensemble de méthodes est défini :
  - **`empty()`**, **`size()`**, **`clear()`**, **`begin()/end()`**, **`rbegin()/rend()`**
- Différents types de conteneurs :
  - conteneurs simples : **`pair`**
  - conteneurs séquentiels > stockent les éléments et y accèdent de manière séquentielle : **`vector`**, **`list`**.
  - conteneur associatifs > associent une clé à chaque objet stocké, ces clefs servent ensuite à accéder efficacement aux objets : **`map`**.
  - adaptateurs de conteneurs > spécialisation de conteneurs : **`deque`** spécialisé en **`queue`** ou **`stack`**.

Les différents conteneurs se caractérisent par l'efficacité de certaines de leurs opérations. Le choix d'un conteneur doit donc être guidé par l'utilisation que l'on souhaite en faire.

Quelques complexités avant de commencer

Différentes complexités ...

Exemples d'utilisations : vector

```
1  #include<iostream>
2  #include<vector> // <-- utilisation des vecteurs
3  using namespace std ;
4  int main(void)
5  {
6     vector<int> v1; // vecteur d'entiers vide
7     vector<int> v2(8) ; // vecteurde 8 cases non initialisees
8     vector<int> v3(8,17); // vecteur de 8 cases initialisees a 17
9     cout << "Taille de v1 : " << v1.size() << endl;
10    // Taille de v1 : 0
11    cout << "Taille de v2 : " << v2.size() << endl;
12    // Taille de v2 : 8
13    cout << "Taille de v3 : " << v2 .size() << endl;
14    // Taille de v3 : 8
15    for(int i = 0 ; i < v3.size(); ++i) {
16            cout << " v3.at( " << i << " ) : " << v3.at( i ) ; // i n d e x a t i o n v i a l a
17            // methode a t ( )
18            cout << " ou v3 [ " << i << " ] : " << v3[ i ] << endl ; // i n d e x a t i o n v i
19            // l e s [ ]
20    }
21    return 0;
22 }
```

Attention : le parcours simple via l'opérateur [] n'est pas disponible pour tous les conteneurs → passage par les itérateurs.

Exemples d'utilisations : matrices avec vector

```
1  #include<iostream>
2  #include<vector> // <-- utilisation des vecteurs
3  using std::cout;
4  using std::endl;
5  typedef vector<float> Line;
6  typedef vector<Line> Matrix;
7  int main(void)
8  {
9     Line l(17, 0.0); // ligne de 17 elements nul s
10    Matrix m(17, l); // matrice de 17 lignes
11    // vector <<vector<float>> m;
12    // --> necessite d'initialiser m "a la main "
13    for(uint i = 0; i < m.size(); ++i)
14    {
15            for (uint j = 0; j < m[i].size(); ++j)
16            {
17                    cout << m[i][j] << " ";
18            }
19            cout << endl;
20    }
21    return 0;
22 }
```

Exemples d'utilisations : list

```
1  #include<iostream>
2  #include<list> // <-- utilisation des listes doublement chainees
3  using std::cout;
4  using std::endl;
5  int main ( v o i d )
6  {
7          list<int> L; // declaration similaire a celle d'un vecteur
8          L.pushback( 5 ); // L = {5}
9          L.pushback( 4 ); // L = {5, 4 }
10         L.pushfront( 3 ); // L = { 3, 5, 4}
11         cout << "Tete de L : " << L .front() << endl; // 3
12         cout << "Queue de L : " << L.back() << endl; // 4
13         for (uint i=0; i < L.size(); ++i)
14         {
15                 cout << "L.at( " << i << " ) : " << L.at( i ) ; // ERREUR
16                 cout << "L[ " << i << " ] : " << L[ i ] << endl ; // ERREUR
17         }
18         return 0;
19 }
```

- la méthode **pushback** est également définie sur les vecteurs.
- pour les **list** l'indexation via l'opérateur **[]** ou **at()** n'est pas définie.

Exemples d'utilisations : map

```
1  #include<iostream>
2  #include<map>// <-- utilisation des tables associatives ( map )
3
4  using namespace std;
5  int main(void)
6  {
7  map<string ,string> telephones; // tableau indexe par une
8  // chaine de caracteres
9  telephones["Nom1"] = "0132655698";
10 telephones["Nom2"] = "0686423657";
11 cout << " Nom1 -> " << telephones["Nom1"] << endl;
12 cout << " Nom2 -> " << telephones["Nom2"] << endl;
13 return 0;
14 }
```

Qu'est ce qu'un itérateur ?

- Problème : certains conteneurs comme les listes chaînées ne stockent pas nécessairement leurs éléments de façon contigüe en mémoire.
- impossible d'utiliser l'arithmétique sur les pointeurs
- afin d'homogénéiser les actions sur les conteneurs, la notion d'itérateur a été introduite
- Un itérateur peut être vu comme un pointeur sur un élément du conteneur. Il permet :
  - de parcourir un conteneur
  - d'accéder aux données contenues dans le conteneur
  - de modifier les données
- Il existe deux types d'itérateurs :
  - iterator ou const iterator → parcours du début à la fin
  - reverse iterator ou const reverse iterator → parcours à l'envers (de la fin au début).

Qu'est ce qu'un itérateur ?

- Tous les itérateurs partagent les propriétés suivantes :
- ils possèdent une valeur qui désigne (pointe vers) un élément donné d'un conteneur.
- ils peuvent être incrémentés via l'opérateur ++ afin de pointer sur l'élément suivant.
- ils peuvent être déréférencés comme un pointeur en utilisant l'opérateur *.
- deux itérateurs sur un même conteneur peuvent être comparés par égalité ou inégalité.
- Certains itérateurs peuvent posséder des propriétés supplémentaires :
  - décrémentation par l'opérateur – $\rightarrow$ itérateur bidirectionnel
  - accès direct : dans ce cas l'opérateur [] est défini et [i] est équivalent à *(it + i) qui a un sens

Utilisation générale d'un itérateur

- Un itérateur associé à un conteneur C (vecteur, liste, ...) se déclare de la manière suivante :

```
C::iterator it1 ;
C::reverse iterator it2 ;
```

```
// Exemple
list<int>::iterator it;
```

- Un iterator peut être initialisé grâce aux méthodes :
  - **`begin()`** : → renvoie un itérateur pointant sur le premier élément du conteneur
  - **`end()`** : → renvoie un itérateur pointant sur l'élément juste après le dernier élément du conteneur
  - tout autre méthode retournant un itérateur sur le conteneur : **`find()`**, ...
- De manière similaire un reverse iterator peut être initialisé avec les méthodes : **`rbegin()`**, **`rend()`**, ...

Exemples d'utilisations : affichage d'une matrice

```
1  #include<iostream>
2  #include<vector>
3
4  // <-- utilisation des vecteurs
5
6  u s i n g namespace s t d ;
7  typedef vector<float> Line;
8  typedef vector<Line>Matrix;
9  int main(void)
10 {
11         Line l(17, 0.0);
12     // declaration d'une ligne de 17 elements nuls
13         Matrix m(17, l); // declaration d'une matrice de 17 lignes
14         Matrix::iterator i;
15         Line::iterator j;
16         for( i = m.begin(); i != m.end(); i++) {
17                 for( j = (*i).begin(); j!=(*i).end(); j++)
18                         cout << ( *j ) << " ";
19                 cout << endl ;
20         }
21         return 0;
22 }
```

Exemples d'utilisations : list

```
#include<iostream>
#include<list> // <-- utilisationdes listes doublement chainees
using namespace std;
int main(void)
{
list<int> L; // declarations imilaire a celle d'un vecteur
L.pushback( 5 ); // L = {5}
L.pushback( 4 ); // L = {5 , 4 }
L.pushfront( 3 ) ; // L = { 3 , 5 , 4}
cout << " Tete de L : " << L.front() << endl; // 3
cout << " Queue de L : " << L.back() << endl ; // 4
int idx = 0;
list<int>::iterator it1;
list<int>::reverseiterator it2;
for (it1 = L.begin(); it1 != L.end(); it1++)
cout << "L(" << (idx++) << " ) : " << ( *it1 ) << endl; // OK
// --> 3 , 5 , 4
for( it2 = L.rbegin(); it2 != L.rend(); it2++)
cout << "L(" << (idx++) << " ) : " << ( *it2 ) << endl;
// --> 4 , 5 , 3
return 0;
}
```

Le type auto

C++11 a ajouté la possibilité de déduire automatiquement le type d'une variable en fonction de son expression d'initialisation à l'aide du mot-clé auto.

```
auto n = 12; // 12 estde type int donc n est de type int .
auto x = 3.5L // x sera de type long double.
auto p = 2* n; // 2*n est de type int donc p aussi.
```

Le qualificatif const n'est pas considéré par le type auto

```
const int n = 12;
a u t o p = n ; // p s e r a de t y p e int e t p a s c o n s t int
```

On peut toutefois rajouter le qualificatif dans la déclaration automatique de type

```
const int n = 12;
c o n s t a u t o p = n ; // ( ou a u t o c o n s t ) p e s t b i e n de t y p e c o n s t
```

Le type auto est surtout utilisé quand le type de la variable est long à écrire par exemple pour les itérateurs.

Pointeurs de fonctions

En C++ il n'est pas possible de placer le nom d'une fonction dans une variable. En revanche on peut définir une variable afin de pointer sur une fonction de la façon suivante :

```
int (*adf)( int , double); // *adf est une fonction a 2
// arguments de type int et double et renvoyant unparametre
// de type int -> adf est un pointeur sur une telle fonction.
```

On peut donc assigner l'adresse d'une fonction à un pointeur sur fonction de même type.

```
int fct1( int, double);
int fct2(int&, double);
int fct3(const int, float);
adf = &fct1; // L'utilisation de & n'est pas obligatoire
adf = fct2; // Erreur int et int&
adf = fct3; // Erreur double et float
```

Pointeurs de fonctions

- On peut programmer un appel de fonction “variable”, c’est à dire que la fonction peut varier au fil de l’exécution du programme via un pointeur de fonction.

```
1  int (*adf)(int, double) = fct1;
2  (*adf)(2, 3.7) ; // Appelde fct1 via adf
```

- L’intérêt des pointeurs de fonction est de permettre l’appel d’une fonction parmi un éventail de fonctions au choix.
  - fonction de rappel
  - appeler une fonction dont on connaît l’indice de son pointeur dans un tableau de pointeurs sur fonction

Expressions lambdas

- Afin notamment de simplifier l'écriture des pointeurs de fonction, C++11 a introduit les expressions lambdas qui permettent de définir une fonction directement à la place de l'argument fonction.
- peut augmenter la lisibilité du code
- Les expressions lambdas permettent également de "capturer" des variables locales même si dans ce cas là elles ne pourront pas être utilisées à la place d'un pointeur de fonction.

```
affiche( f1 );
affiche( [] ( double x ){ return x*x; } );
affiche( [] ( double x ) {
double y = 3* x * x -2;
if ( y < 0)
y = -y +2;
return y;
});
```

```
[] (double x){return x*x;} //est une expression lambda.
```

Expressions lambdas

- Il est possible de nommer une expression lambda afin de la stocker dans une variable :

```
1  auto lambda = [](double x ){ return x * x; };
2  cout << lambda(5.25) << endl ; // Affiche 27.56
```

- Le `[]` définissent la liste de capture de l'expression lambda (dans ce cas vide).
- Des variables locales peuvent être inclues dans la liste de capture, ainsi l'expression lambda recevra une copie de la valeur au moment ou elle sera créée.
- Les variables locales peuvent être passées par référence auquel cas la distinction utilisation/création n'aura plus d'importance.

Expressions lambdas

```
1   #include<iostream>
2   using namespace std;
3   int main( )
4   {
5           int n=5;
6           auto f1[n]() { cout << " Valeur de n dans f1 : " << n << endl; };
7           auto f2[&n]() { cout << " Valeur de n dans f2 : " << n << endl ; };
8           f1(); //
9           f2(); //
0           n = 20;
11          f1(); //
2           f2(); //
3
4   n est capture par valeur
5   n est capture par reference
6   n est capture par la valeur
7   n est capture par reference
8
9   return 0;
0   }
```

Exemples d'algorithmes

- La STL offre un ensemble d'algorithmes utilisables sur les conteneurs, via les itérateurs.
- Les principaux algorithmes sont :
  - les algorithmes numériques : minimum, maximum, sommes partielles, accumulations, produits, . . .
  - les algorithmes modifiant les conteneurs : remplissage, copie, échanges, union, insertion, . . .
  - les algorithmes ne modifiant pas les conteneurs : recherche, applications de fonctions, inclusion, . . .
  - les algorithmes de tri

Exemples d'algorithmes : sommer les éléments d'un vecteur d'entiers

```
1  #include<iostream>
2  #include<vector> // <-- utilisation des vecteurs
3  #include<numeric> // <-- utilisation des algorithmes numeriques
4  using namespace std;
5  int main()
6  {
7          vector <int> v1(8); // declarationd'un vecteur de8 cases
8      // non initialisees
9          int idx = 1; // --> initialisation du debut a la fin du vecteur
0
11         for( auto it1 = v1.begin(); it1 != v1.end() ; ++it1)
2                  (*it1) = (idx++);
3      // --> calcul e de lasomme des elements : 1 + 2 + . . . + 8
4          int sum = accumulate( v1.begin(), v1.end(), 0);
5          cout << "Sum : " << sum << endl ;
6          return 0;
7  }
```

Exemples d'algorithmes : trier les éléments d'un vecteur d'entiers

…

```
#include<algorithm> // <-- utilisation des algorithmes de tri
using namespace std;
int main ( ) {
        vector<int> v1( 8 );
        for( auto it1 = v1.begin() ; it1!= v1.end(); it1++)
                (*it1) = rand() nil 101; // 32 32 54 12 52 56 8 30
        for ( it1 = v1.begin(); it1 != v1.end(); it1++)
                cout << ( *it1 ) << " "; // 32 32 54 12 52 56 8 30
        cout << endl ;
        sort ( v1.begin(), v1.end() ); // Tri( utilise < par defaut )
        for( it1 = v1.begin() ; it1 != v1.end(); it1++ )
                cout << ( *it1 ) << " " ; // 8 12 30 32 32 52 54 56
        cout << endl ;
    // Tri avec une fonction de comparaison personalisee
        sort( v1.begin(), v1.end(), []( int a, int b) { return a>b; } );
        for( it1 = v1.begin(); it1 != v1.end(); it1++)
                cout << ( *it1 ) << " "; // 56 54 52 32 32 30 12 8
        cout << endl;
        return 0;
}
```